

C200 PROGRAMMING ASSIGNMENT № 10

Dr. M.M. Dalkilic

Computer Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

December 8, 2024

Introduction

In this homework, you'll work on translating critical thinking to programming. This homework is a bit less intense than the last one! :) In this homework, you will be reading some on your own and experimenting.

Make sure to submit to Autograder and push your work by **December 14th, 11:00PM EDT, Saturday**

1: Numismatics

An unscrupulous collector of coins claims to have n of a particular type. He is lying. There is one fake. You don't have time to scrutinize each coin. Using a balance you can efficiently discover the fake coin (Fig. 1).



Figure 1: Scale balance for three bags.

1. (base case) If there are only three coins x, y, z , then this returns the value:

```
1 (x==y)*z + (y==z)*x + (x==z)*y
```

which, in words, says if x, y weight the same, the fake is z and so on.

2. (inductive case) separate the coins into three equal piles. There obviously might be one or two coins, since n might be divisible by three or have a remainder of 1 or 2.

```
1 >>> 20 % 3, 21 % 3, 22 % 3
2 (2, 0, 1)
```

Two bags must have the same weight. The third bag has the fake. So you split the third bag into three piles (including the 1 or 2 extra if they exist) and repeat the process.

You'll be given a list of coins. Each real coin is worth 1. The fake coin is worth a different positive integer value. For example:

```
1 coins = [1,1,1,1,1,1,1,1,1,1,1,1,1,1,2,1]
```

You'll complete the *recursive find* that takes a list of coins and returns the fake. When you're done, we'll use $\langle list \rangle.index(x)$ method to find the index of the fake coin. The scale is called by the function `weight(x,y)`:

```

1 cnt = 0
2
3 def weigh(x,y):
4     global cnt
5     cnt+=1
6     return sum(x) == sum(y)

```

This is the only function you can use in the code unless there are *exactly* three coins. The global `cnt` allows us to update the global variable locally. The following code:

```

1 coins = [1,1,1,1,1,1,1,1,1,1,1,1,1,2,1]
2 print(coins)
3 fake = find(coins)
4 print(fake, coins.index(fake), cnt)

```

produces:

```

1 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1]
2 2 13 2

```

The fake coin is 2, its index is 13, and we used the balance scale twice. How can that be possible? We'll show the successive weighings:

```

1 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1] →
2 [1,1,1,1,1][1,1,1,1,1] [1, 1, 1, 2, 1] [] →
3 [1,1][1,1] [1, 2] [1] →
4 2 13 2

```

There were two weighings. The `cnt` variable keeps track of the number of weighings.

Programming Problem 1: Fake Coin

- Complete the functions. The function `find` is recursive.
- Do not modify `weigh`.
- `split_weight` takes a non-empty list of at least 3 numbers and returns four lists, the fourth can be empty, contain one, or contain two numbers. For example,

```
1 print(split_weight([1,1,1,1,1,2,1,1]))
```

produces

```
1 ([1, 1], [1, 1], [1, 2], [1, 1])
```

Problem 2: Hieroglyphics

Hieroglyphics was the system of writing in Ancient Egypt. The knowledge that it was writing was lost for many centuries. We now have some strong sense of the sounds of most of the symbols.

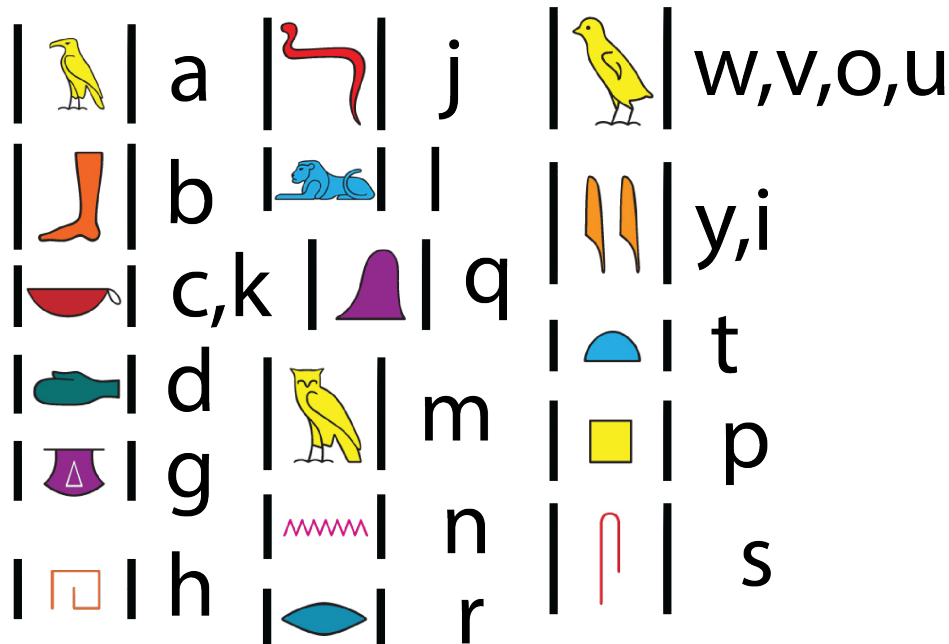


Figure 2: There are the hieroglyphs and approximate letter equivalents. Some of the letters in your name will not be present, so your translation will have to ignore them.

Figure3 shows one list of hieroglyphs and sounds. As you can observe, not all sounds in English have a corresponding hieroglyph. This program will read your name from a file, then translate it to the appropriate symbols and create an HTML page with the cartouche (surrounding ellipse) indicating it's a formal, important name.

Each of the symbols has a corresponding jpg image with the name `eglyph $x$` where x is the letter. The top and bottom of the cartouche are `eglyphTOP.jpg`, `eglyphBOTTOM.jpg`, respectively. You'll finish a class for building HTML pages and a couple of function. The following code:

```
1 page = my_HTML()
2 print(page)
3 page.add_element(f"{name} = {translate(name)}", element = 'h2' )
4 print(page)
5 page = page + "eglyphTOP.jpg"
6 add_hieroglyphs(page,name)
7 page = page + "eglyphBOTTOM.jpg"
8 print(page)
9 page.save(translate(name))
10
11 webbrowser.open_new("" + translate(name) + ".html")
```

The HTML is nothing more than a simple string.

- Line 1 creates an instance of the class.
- Line 2 prints the current page. This is a blank page (look at the output).
- Line 3 adds the H2 header. Which displays the first line of the webpage.
- Line 4 prints it.
- Line 5 uses an overloaded + to add an image. This adds the string:

```
1 <img src='eglyphTOP.jpg' class='center'><br>
```

given a string that describes the file.

- Line 6 is a function that takes a webpage and a name and adds the image tags as file-names to the page.
- Line 9 is a method that takes a string and writes the HTML (string) to that location.

adds h2 element. As you can see we're adding the name = translate(name) as a string. We're using a switch to indicate this must be the h2 tag. produces:

```
1 <html><head></head><body></body></html>
2
3 <html><head></head><body><h2 class='text-center'> Mehmet = mhmt </h2>
4 </body></html>
5
6 <html><head></head><body><h2 class='text-center'> Mehmet = mhmt </h2>
7 <img src='eglyphTOP.jpg' class='center'><br>
8 <img src='eglyphm.jpg' class='center'><br>
9 <img src='eglyphh.jpg' class='center'><br>
10 <img src='eglyphm.jpg' class='center'><br>
11 <img src='eglypht.jpg' class='center'><br>
12 <img src='eglyphBOTTOM.jpg' class='center'><br>
13 </body></html>
```

The last line opens a local webpage with my name:

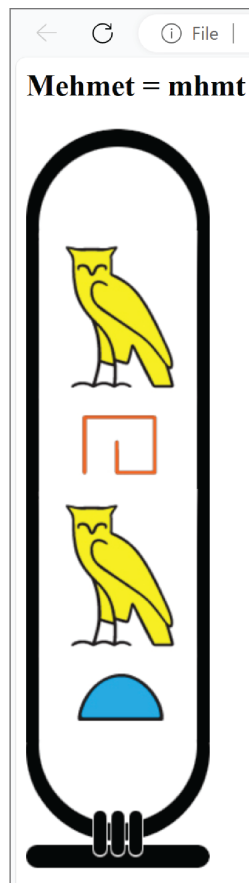


Figure 3: Heiroglyphs and approximate sounds.

Deliverable for Problem 2: The Mummy

- Complete the class. I've finished the init class and string class.
- You'll have your name in a file name.txt that's read in and translated.
- You may need to install webbrowser
- Observe that some letters in a name will be ignored, since there isn't any corresponding hieroglyph. In that case you'll have to ignore the letter.
- The translate function should change to lower case.
- All the files are given and you should look at them.
- The save uses your Egyptian name as the filename.html.
- I'll include the file I created.

Problem 3: Decision Trees From a Relation

As we saw in lecture, one of the most successful techniques in AI is building a function $f : X \rightarrow Y$. We are given data in the form

$$data = \{((x_{00}, x_{01}, \dots, x_{0m}), y_0), ((x_{10}, x_{11}, \dots, x_{1m}), y_1), \dots, ((x_{n0}, x_{n1}, \dots, x_{nm}), y_n)\} \quad (1)$$

as a relation: where TID is the tuple ID and each row is called a *tuple*. X collectively are known

TID	X_0	X_1	$\dots$	X_m	Y
0	x_{00}	x_{01}	$\dots$	x_{0m}	y_0
1	x_{10}	x_{11}	$\dots$	x_{1m}	y_1
$\vdots$	$\vdots$				
n	x_{n0}	x_{n1}	$\dots$	x_{nm}	y_n

as the *features* and Y the label or class label or class. In database parlance X, Y are known as attributes. Given the relation (or data) we build a *decision tree* that's composed from a series of features and values. The function we'll be building in this problem is for this data:

- A Person experienced 90 days past due delinquency or worse as 0/1 (Boolean)
- B Total balance on credit cards and personal lines of credit except real estate and no installment debt like car loans divided by the sum of credit limits as percentage (float)
- C Age of borrower in years as integer
- D Number of times borrower has been 30-59 days past due but no worse in the last 2 years as integer
- E Monthly debt payments, alimony, living costs divided by monthly gross income as percentage (float)
- F Monthly income as float
- G Number of Open loans (installment like car loan or mortgage) and Lines of credit (e.g. credit cards) as integer
- H Number of times borrower has been 90 days or more past due integer as
- I Number of mortgage and real estate loans including home equity lines of credit as integer
- J Number of times borrower has been 60-89 days past due but no worse in the last 2 years as integer
- K Number of dependents in family excluding themselves (spouse, children etc.) as integer

Here are three tuples: So, the first tuple indicates the person was delinquent of at least a

A	B	C	D	E	F	G	H	I	J	K
1	0.766126609	45	2	0.802982129	9120	13	0	6	0	2
0	0.957151019	40	0	0.121876201	2600	4	0	0	0	1
0	0.65818014	38	1	0.085113375	3042	2	1	0	0	0

month (Feature A's definition), but the other two were not. We want to build a function $f : BCDEFGHIJK \rightarrow A$ so that when we see a person's history:

$$[0.025892469, 38, 0, 74.5, 1, 13, 0, 0, 0, 2] \quad (2)$$

$$[0.762158561, 44, 4, 0.224465502, 21000, 5, 0, 1, 0, 3] \quad (3)$$

we can predict if there will be a likely delinquency at least a month. A decision tree will build a structure that will try to solve this problem. We will have spoken about this in lecture, but you should visit: <https://scikit-learn.org/stable/modules/tree.html> and run the example with the iris data with which you are familiar. The tree can be visualized:

The following code:

```
1 tree_model = build_tree_model("cr.csv")
2 unknown = pd.DataFrame(data = [[0.025892469, 38, 0, 74.5, 1, 13, 0, 0, 0, 2],
3                               [0.762158561, 44, 4, 0.224465502, 21000, 5, 0, 1, 0, 3]], ←
4                               columns = ['B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K'])
5 print(tree_model.predict(unknown)) #1,0
6 tree.plot_tree(tree_model)
7 plt.show()
```

produces:

```
1 [1 0]
```

which means the first person will, according to the data, be delinquent, and the second one will not. Your function will take a string (name of the file), using pandas do the following:

- Clean the data by removing rows that have Na (not a number). We covered this in lecture, but as hint, look for dropna.
- Separate the table into two data frames: B-K (which are the features) and A which is the label or target. You build the tree shown in the url.
- Return the the fitted tree model

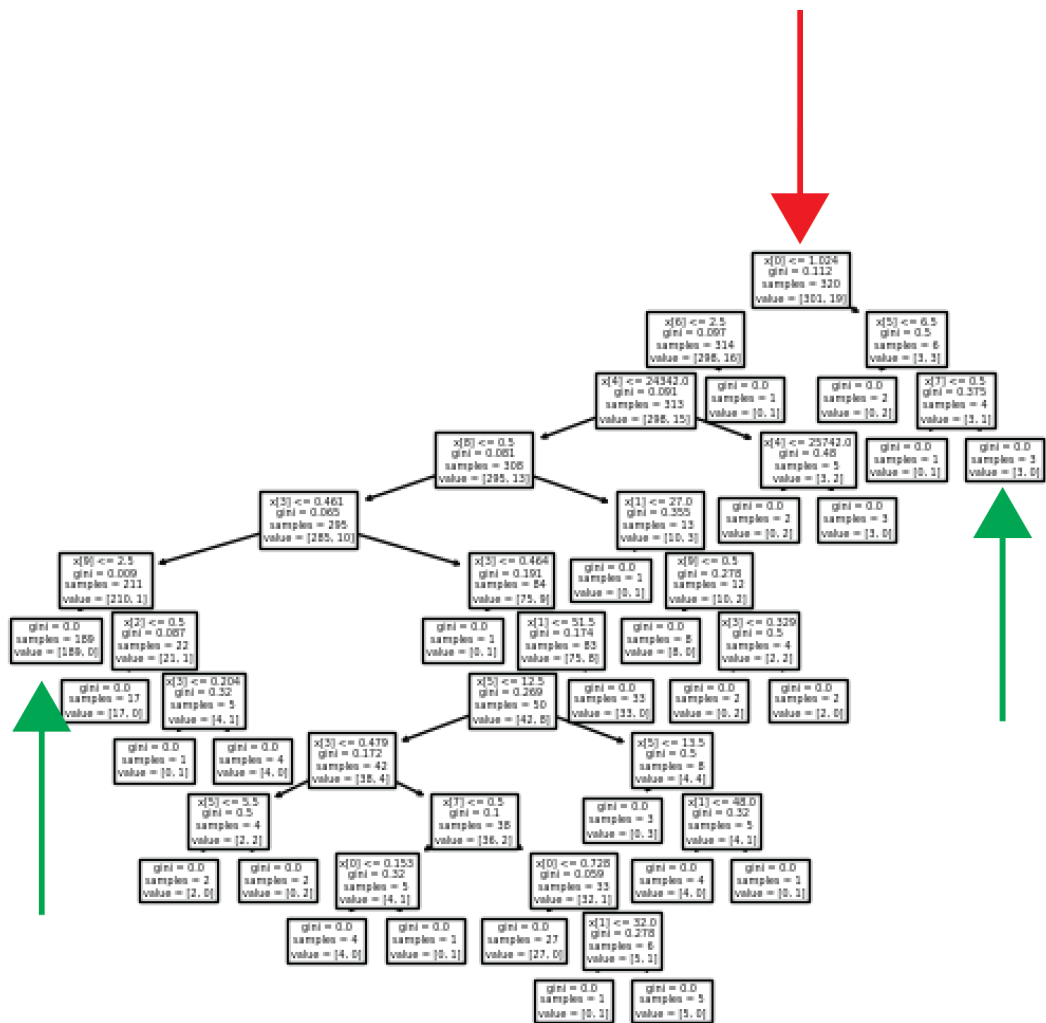


Figure 4: Red points to where data starts (root); green points to a possible end (leaf).

Decision Tree

- Implement the `build_tree_model`. The tasks are described above, but your task is to return the fitted model.
- Confirm that your prediction is 1,0 for the pairs of unknown, respectively.
- If your model is build correctly, you'll be able to visualize it.
- Remember there was a condition for dropna to be committed.

4: Näive Bayes In A Simpler Setting

In this problem, we'll look at what's called *maximum likelihood*. We saw in Näive Bayes that the likelihood was $P(\text{data}|\text{class})$. We needed some machinery to consider the prior. Here we dispense with the prior and simply return the maximal likelihood.

Assume you observe someone flipping a coin. The flips are HHTT. Here are four possible models: We have a method of selecting the best model by picking the one that has the greatest

model	$P(H)$	$P(T)$
0	.1	.9
1	.3	.7
2	.5	.5
3	.6	.4
4	.8	.2

Table 1: $P()$ is the probability of heads or tails.

likelihood. Let v be the product of the probabilities of the throws:

$$v(2) = P(H) \cdot P(H) \cdot P(T) \cdot P(H) \quad (4)$$

$$= \left(\frac{1}{2}\right)^4 = \frac{1}{16} = 0.0625 \quad (5)$$

where 2 corresponds to the third entry (equal chances) corresponding to the flips HHTT. We can check each of the four models:

$$v(0) = (.1)^2(.9)^2 = 0.0081 \quad (6)$$

$$v(1) = (.3)^2(.7)^2 = 0.0441 \quad (7)$$

$$v(2) = 0.0625 \quad (8)$$

$$v(3) = 0.0576 \quad (9)$$

$$v(4) = 0.0256 \quad (10)$$

with explicitly the flip probabilities shown. We select the model that gives the greatest value: 2. This makes intuitive sense because the number of flips has half heads and half tails.

The following code:

```
1 models = {0:{ "H":.1, "T":.9},1:{ "H":.3, "T":.7},2:{ "H":.5, "T":.5},
2           3:{ "H":.6, "T":.4},4:{ "H":.8, "T":.2}}
3 flips = "HHTT"
4 print(likelihood(flips,models[2]))
5
6
7 flips = "H"*rn.randint(0,5) + "T"*rn.randint(0,5)
8 print(flips)
9 print(max_likelihood(flips,models))
```

produces:

```
1 0.0625
2 HTTTTT
3 [(0, 0.059049000000000002), (1, 0.050420999999999999),
4  (2, 0.015625), (3, 0.006144000000000001),
5  (4, 0.0002560000000000001)]
```

The function `likelihood` returns the product of the flips given a specific model. The function `max_likelihood` returns a sort list descending of pairs `x,y` where `x` is the model and `y` is the product.

Coin Flips

- Implement the two functions.

Programming Partners

lddial@iu.edu, aposton@iu.edu
laigrant@iu.edu, tersacks@iu.edu
adatwani@iu.edu, slopezve@iu.edu
cjo3@iu.edu, joevu@iu.edu
meiyhe@iu.edu, ndmyer@iu.edu
alurs@iu.edu, bsati@iu.edu
adj18@iu.edu, rnalluri@iu.edu
jamcham@iu.edu, shwanara@iu.edu
okirchen@iu.edu, auskirvi@iu.edu
jbriar@iu.edu, taylbryo@iu.edu
agahlau@iu.edu, nshnoble@iu.edu
yaljabri@iu.edu, ansjmeht@iu.edu
yc127@iu.edu, audscha@iu.edu
nantinao@iu.edu, rvasude@iu.edu
ttfender@iu.edu, ishyadav@iu.edu
iyersan@iu.edu, cmellgre@iu.edu
ajafowl@iu.edu, jopantoj@iu.edu
enkarl@iu.edu, pjminne@iu.edu
jhl14@iu.edu, smitaa@iu.edu
keswa@iu.edu, youngct@iu.edu
mdevara@iu.edu, neenpate@iu.edu
rbernfel@iu.edu, camoverm@iu.edu
ealfalah@iu.edu, hsh1@iu.edu
jjithesh@iu.edu, matyun@iu.edu

vkabra@iu.edu, jacloise@iu.edu
ghargro@iu.edu, jwubelho@iu.edu
mjfarrin@iu.edu, kenmoses@iu.edu
cedrchen@iu.edu, nmenne@iu.edu
damaatki@iu.edu, gl25@iu.edu
nifredri@iu.edu, pmiesch@iu.edu
mahichan@iu.edu, thebmill@iu.edu
dajuhnke@iu.edu, mmalviya@iu.edu
rdotsu@iu.edu, anupand@iu.edu
tbashee@iu.edu, kiloscot@iu.edu
ckbarts@iu.edu, jnimmala@iu.edu
jejea@iu.edu, sahasrir@iu.edu
ojallow@iu.edu, muksingh@iu.edu
asgurram@iu.edu, rvarrier@iu.edu
nafike@iu.edu, everma@iu.edu
zkaacin@iu.edu, bawsung@iu.edu
skalluru@iu.edu, whitedyl@iu.edu
srkagama@iu.edu, zs15@iu.edu
jigarnes@iu.edu, davepais@iu.edu
ethdeatr@iu.edu, pp22@iu.edu
clickc@iu.edu, dmonroyh@iu.edu
abondada@iu.edu, jskains@iu.edu
florech@iu.edu, jiajtang@iu.edu
jb108@iu.edu, aluckhau@iu.edu
bdgonzal@iu.edu, sxanders@iu.edu
jbosio@iu.edu, cs155@iu.edu
bthenson@iu.edu, wl52@iu.edu
adhimat@iu.edu, ozsachs@iu.edu
ethhanna@iu.edu, aamuse@iu.edu
mthinman@iu.edu, kpazdur@iu.edu
evarcarm@iu.edu, jjrinald@iu.edu
grhamlin@iu.edu, westje@iu.edu
natkling@iu.edu, mitcal@iu.edu
jagoodal@iu.edu, smitfi@iu.edu
benchart@iu.edu, bogawa@iu.edu
eabilius@iu.edu, jl367@iu.edu
drussow@iu.edu, cpmann@iu.edu
talfares@iu.edu, mmaplet@iu.edu
mrb9@iu.edu, cjrodenb@iu.edu
ldeel@iu.edu, dyllopez@iu.edu
mfbah@iu.edu, isundara@iu.edu

rgyasini@iu.edu, aidsteph@iu.edu
dbarbari@iu.edu, emasseng@iu.edu
ygajjar@iu.edu, rheashah@iu.edu
agotanco@iu.edu, cmekat@iu.edu
langarbe@iu.edu, sumuth@iu.edu
clarkina@iu.edu, roldenbu@iu.edu
ehchoo@iu.edu, esspatel@iu.edu
rkoston@iu.edu, mkreddy@iu.edu
pranchan@iu.edu, anryoun@iu.edu
ljhorman@iu.edu, stewrach@iu.edu
ebederma@iu.edu, anrajput@iu.edu
eberens@iu.edu, mmmatias@iu.edu
vicbarre@iu.edu, irodeman@iu.edu
jaeblank@iu.edu, kevwong@iu.edu
nwconn@iu.edu, jrwaren@iu.edu
coubenne@iu.edu, klohiya@iu.edu
btcunnin@iu.edu, gsackie@iu.edu
tycarmo@iu.edu, drimawi@iu.edu
ldeveau@iu.edu, rasahu@iu.edu
egilmour@iu.edu, ssp4@iu.edu
ramgowda@iu.edu, jvmenach@iu.edu
zahaider@iu.edu, kr15@iu.edu
bedani@iu.edu, hmahapat@iu.edu
carthugh@iu.edu, ssanjee@iu.edu
mgoenka@iu.edu, martibr@iu.edu
efoltyn@iu.edu, clushell@iu.edu
edeitchl@iu.edu, hailreid@iu.edu
jkittur@iu.edu, kalovert@iu.edu
dayinla@iu.edu, tlichten@iu.edu
felkhatt@iu.edu, hshyama@iu.edu
mugabr@iu.edu, ctsvetan@iu.edu
timolaws@iu.edu, conaitis@iu.edu
agrief@iu.edu, omwells@iu.edu
alneyadk@iu.edu, sstickl@iu.edu
bcarpine@iu.edu, dematt@iu.edu
bgelard@iu.edu, tunnitha@iu.edu
nimjain@iu.edu, soyedele@iu.edu
lafly@iu.edu, nmeely@iu.edu
sdondeti@iu.edu, negia@iu.edu
nbriere@iu.edu, aramanu@iu.edu
evbutts@iu.edu, robreese@iu.edu

banksda@iu.edu, missavin@iu.edu
crigonza@iu.edu, vivitran@iu.edu
mahchaud@iu.edu, ibshaw@iu.edu
ssdatla@iu.edu, ijrussel@iu.edu
skapure@iu.edu, marsh1@iu.edu
rogoenka@iu.edu, isvoor@iu.edu
adguhan@iu.edu, jawegman@iu.edu
abrking@iu.edu, lukthiel@iu.edu
ahmealsh@iu.edu, maymat@iu.edu
akeshav@iu.edu, sasubb@iu.edu
ahsans@iu.edu, djtheodo@iu.edu
dh43@iu.edu, jasreese@iu.edu
heoji@iu.edu, roberjuv@iu.edu
jk314@iu.edu, jmsequei@iu.edu
ndodoh@iu.edu, prnemani@iu.edu
bronburk@iu.edu, mzuhair@iu.edu
dgiffor@iu.edu, sunathan@iu.edu
sydforem@iu.edu, amarynow@iu.edu
chjflor@iu.edu, elwesley@iu.edu
mgbekele@iu.edu, antonegr@iu.edu
ajkouts@iu.edu, tnerkar@iu.edu
chfurlow@iu.edu, aishravi@iu.edu
akoduru@iu.edu, marcmarq@iu.edu
jergalan@iu.edu, navann@iu.edu
skyangel@iu.edu, aralover@iu.edu
concraft@iu.edu, ozopich@iu.edu
mailic@iu.edu, jw363@iu.edu
am280@iu.edu, rvenigal@iu.edu
drmiguth@iu.edu, vpremku@iu.edu
pchitwoo@iu.edu, jakpatel@iu.edu
ag38@iu.edu, kokulski@iu.edu
jkuk@iu.edu, braxluns@iu.edu
nicdelg@iu.edu, evanmayo@iu.edu
nikdevra@iu.edu, mjo4@iu.edu
idasilva@iu.edu, ksabloak@iu.edu
keswar@iu.edu, risvphil@iu.edu
mbenites@iu.edu, sholove@iu.edu
mabana@iu.edu, kalerobi@iu.edu
dakomi@iu.edu, fsafianu@iu.edu
jaccryst@iu.edu, adivaidy@iu.edu
boydmad@iu.edu, nguyen17@iu.edu

jdkakare@iu.edu, ainlukas@iu.edu
btbasing@iu.edu, ydpatil@iu.edu
nfarzane@iu.edu, athason@iu.edu
kevgross@iu.edu, lydiwang@iu.edu
owencape@iu.edu, lisalaza@iu.edu
xmdrew@iu.edu, dajshaw@iu.edu
joglickm@iu.edu, mawlorch@iu.edu
addfishe@iu.edu, adenyu@iu.edu
shkami@iu.edu, olesmith@iu.edu
kadlakha@iu.edu, liaobrie@iu.edu
sabyrap@iu.edu, ncw1@iu.edu
etferba@iu.edu, zz67@iu.edu
smhamrah@iu.edu, atsarver@iu.edu
byersjac@iu.edu, aidpride@iu.edu
blbinder@iu.edu, rpolu@iu.edu
salvchri@iu.edu, prmundra@iu.edu
radutta@iu.edu, isapittm@iu.edu
iblay@iu.edu, fasrasoo@iu.edu
natkwon@iu.edu, mszach@iu.edu
heissler@iu.edu, corawang@iu.edu
rycostin@iu.edu, vnathany@iu.edu
tborder@iu.edu, adimaran@iu.edu
cgcheane@iu.edu, bgtruong@iu.edu
maboncos@iu.edu, cardnorr@iu.edu
sergonza@iu.edu, tkthang@iu.edu
armstan@iu.edu, samyadav@iu.edu
cdasari@iu.edu, kzerbino@iu.edu
gkreutes@iu.edu, reshetty@iu.edu
aabdulw@iu.edu, froy@iu.edu
herncr@iu.edu, rosanghv@iu.edu
aare@iu.edu, jakepars@iu.edu
blstcoop@iu.edu, mt87@iu.edu
nitiarun@iu.edu, zhangsuo@iu.edu
michdool@iu.edu, nealsing@iu.edu
reclay@iu.edu, vpottete@iu.edu
garmenda@iu.edu, angzamor@iu.edu
kbussel@iu.edu, wschrama@iu.edu
adagar@iu.edu, alzemlya@iu.edu
adekolaw@iu.edu, manpate@iu.edu
zalsamri@iu.edu, mahonm@iu.edu
hdjour@iu.edu, mjseymou@iu.edu

mkulekci@iu.edu, msharsh@iu.edu
aarepal@iu.edu, seuryu@iu.edu
adodaro@iu.edu, sandro@iu.edu
pikhatri@iu.edu, cpmathew@iu.edu
xbchrist@iu.edu, anangla@iu.edu
elscheng@iu.edu, lemarc@iu.edu
twchase@iu.edu, tanilitt@iu.edu
jahjacks@iu.edu, cjlomax@iu.edu
bahls@iu.edu, olsoncs@iu.edu
cammlanc@iu.edu, cmolaski@iu.edu
jackchap@iu.edu, dommonte@iu.edu